

JalSetu

Water Bridge — a smart, conjunctive canal-and-groundwater system for the tail end of the Lower Ganga Canal, Prayagraj

A mechanical engineering student's account of finding a real problem at home, and building a first answer to it



STEP 1 OF 6 · THE PROBLEM



When the Canal Runs Dry, the Aquifer Pays for It

WHAT'S ACTUALLY GOING ON

- When the Lower Ganga Canal falls short at the tail end, farmers do the only thing they can — they drill a tubewell and pump groundwater instead.
- Nobody's really managing this as one system. The Irrigation Department runs the canal, the Groundwater Department/CGWB watches the aquifer, and the two never actually talk to each other.
- Prayagraj is already flagged as a groundwater-mining hotspot. There's no local sensor watching it closely enough to catch trouble before it sets in.

WHO FEELS THIS, AND WHY IT MATTERS

- Tail-end farmers are the ones who actually pay for this — in lost yield and in diesel, every single time the canal comes up short.
- Irrigation engineers can't fairly police a network that stretches roughly 8,183 km by hand — nobody could.
- Groundwater is already carrying most of UP's irrigation load, so a canal shortfall isn't a separate problem — it's more pressure on the same aquifer everyone shares.

Why I picked this problem: the Lower Ganga Canal's own tail district is Kaushambi, right next to Prayagraj. This isn't a case study I picked off a list — it's the ground I actually live on.

The Missing Piece Wasn't Water — It Was Data

84%

of UP's irrigation already leans on groundwater and tubewells, not canals

8,183 km

of branch canal in the LGC network — far too much to watch by hand

0%

of canal and groundwater data is actually shared between the two departments running it

THE INSIGHT THAT CHANGED MY DIRECTION

I went in expecting a water-scarcity problem. What I actually found was an information gap: two departments quietly running one shared hydrological system with no data passing between them. Fixing that data bridge matters more than any single new pump or pipe would.

WHAT I READ TO GET HERE

- CGWB's Dynamic Groundwater Resources Assessment — this is what confirmed Prayagraj as a mining-hotspot district, not just my own assumption.
- CSE's State of Groundwater in Uttar Pradesh — the source behind the 84% tubewell-dependency figure above.
- UP Irrigation Department's own canal records — these corrected an early assumption of mine. I'd initially assumed other UP hotspots like Meerut and Agra sat on this same canal network. They don't — they're on entirely separate systems.

STEP 3 OF 6 · THE SOLUTION



Teaching the System to See a Shortage Coming

JalSetu links canal-flow sensors and groundwater piezometers to a forecasting model that predicts a deficit 24–48 hours in advance — so a gate can be adjusted before the tail end actually runs dry, instead of after.



Where the AI actually sits

A time-series LSTM model reads canal discharge, groundwater depth, and weather data, and outputs a single Predicted Deficit Index — the number everything downstream reacts to.



What makes it usable, not just clever

Gates open proportionally, never just on/off. A person can always override the AI. And if the network drops, it quietly falls back to safe local rules instead of freezing up.

TOOLS I ACTUALLY BUILT WITH

Python

HTML / CSS / JS

LSTM (time-series forecasting)

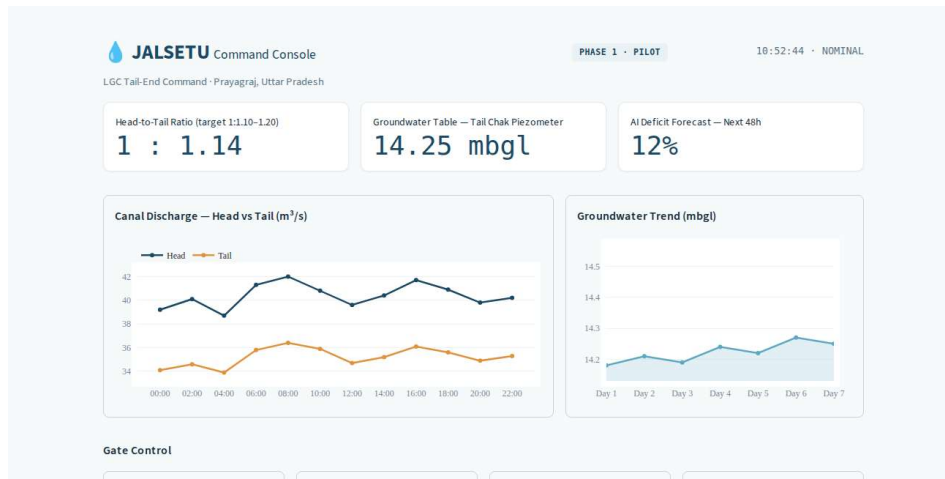
LoRaWAN + MQTT (IoT layer)

The full-scale design (from earlier stages of this project) also specifies the physical IoT layer — non-contact radar/Doppler sensors and solar-powered regulators. What you'll see demonstrated here is the software decision layer, built end-to-end in Python and plain web code.

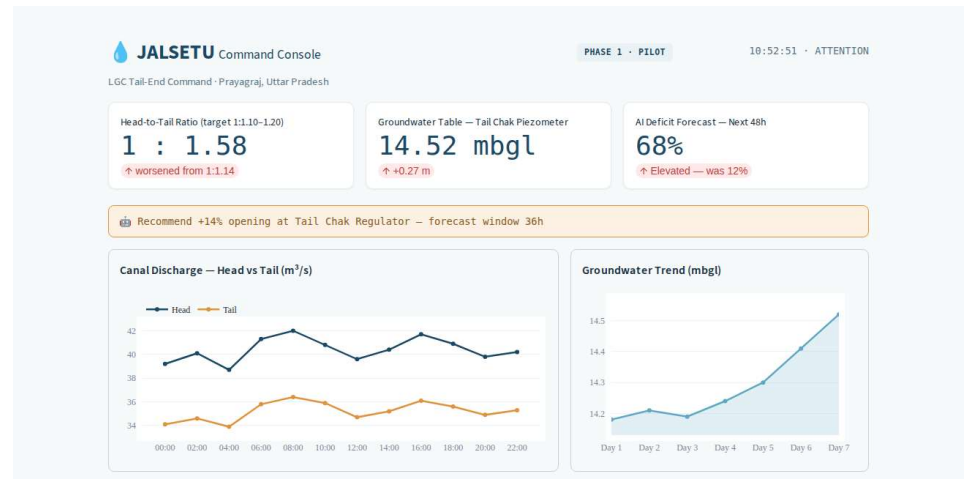
STEP 4 OF 6 · PROTOTYPE



A Command Console You Can Actually Click Through



NORMAL OPERATION



AFTER “SIMULATE DRY-SPELL EVENT”

How you'd actually use it: a JE engineer glances at head-to-tail ratio, groundwater depth, and the AI's forecast in one view. Click Simulate Dry-Spell Event and you can watch it think — flag the risk, recommend a gate change, and log it, live. Switch any gate to manual, and you're back in control instantly.

STEP 5 OF 6 · IMPACT



I Didn't Just Explain the Logic — I Tested It

Scenario	Canal Reading	Groundwater Reading	Gate Response
Canal full, groundwater normal	Full (near sensor)	Normal	0% — closed
Canal empty, groundwater normal	Empty (far from sensor)	Normal	100% — fully open
Canal mid-level, groundwater normal	Midpoint	Normal	~46% — proportional
Canal empty, groundwater stressed	Empty	Stressed	100% — capped at max
Canal full, groundwater stressed — the one that mattered most	Full	Stressed	14% — opens anyway

Why that last row is the whole point: a system watching only the canal would leave that gate shut, because the canal alone looks fine. JalSetu opens it anyway, reacting straight to the aquifer signal — which is really the entire reason this project exists.

WHERE I GOT STUCK, AND WHAT I DID ABOUT IT

- I don't have a real sensor in the ground yet — so before trusting this logic with hardware, I first proved it against the simulated scenarios above.
- An early draft of mine used a groundwater-decline number I'd assumed rather than looked up. It didn't match CGWB's actual data, so I went back and corrected it instead of leaving it in.

STEP 6 OF 6 · WHAT'S NEXT

Where I'd Take This If I Kept Going



Next Milestone

A real Phase 1 pilot on 1–2 distressed minors in Prayagraj, covering roughly 200 ha — putting actual gates on the same decision logic tested here.



Support I'd Need

An actual introduction to the UP Irrigation Dept. or Groundwater Dept./CGWB, and a way to plug into funding already committed through Atal Bhujal Yojana and PMKSY.



How It Could Scale

Across other Lower Ganga Canal districts, across new user groups like FPOs, insurers, and Gram Panchayats — and through the same partnership route above.

THE JOURNEY, IN SIX STEPS

- 1 Problem** Found a real, local gap in Prayagraj
- 2 Discovery** Realized the gap was data, not water
- 3 Solution** Designed an AI layer to see it coming
- 4 Prototype** Built it and got it actually working
- 5 Impact** Tested the logic against 5 scenarios
- 6 Next** Mapped out a real Phase 1 pilot